AI Pioneers

CCA-F | Module 1

Lab 1.2

Controlling Execution: Hooks, Decomposition & Session State

Scenario: AI SOC Copilot for a Financial-Services Firm

Module	M1 — Agentic Architecture & Orchestration
Lab type	Optional · Instructor-Led
Duration	~60 minutes (hands-on)
Environment	Blue Labs VM · Claude Code CLI · Python 3
Sections	S5 (Hooks & Interception) S6 (Task Decomposition Strategies) S7 (Session Management & State Handling)

Lab Objectives

By the end of this lab you will be able to:

- **Implement PostToolUse hooks** that intercept every tool call between the agent's decision and the actual side-effect, so each call can be logged, validated, or blocked — e.g., a SOC copilot stopping a quarantine of a production trading server before it kills market access.
- **Choose between fixed and adaptive decomposition** based on whether the task's shape is known in advance — e.g., a fixed morning threat-intel digest that always runs the same three steps, vs. an adaptive alert-triage router that branches to different playbooks per alert type.
- **Manage long-running session state** via save / resume, fork, and structured summarization — e.g., a SOC investigation that survives a shift change, splits into two competing hypotheses (insider vs external APT), and stays small as it grows past 60 turns.

1. Overview

Lab 1.1 gave you the agentic loop and the coordinator–subagent pattern. Lab 1.2 adds the three controls a production-grade agent needs to operate on real systems without hurting them: guardrails between decision and execution (hooks), the right shape of work for the right kind of task (decomposition), and explicit memory that outlives any single turn (session state). You will build all three around a financial-services Security Operations Center scenario where mistakes carry real cost.

SN	Section	What you will build	Core concept
Ex 1	**S5 — Hooks & Interception**	A hook chain (log / validate / block) wired into a live agentic loop. The agent has tools to quarantine hosts, block IPs, and disable users — the hooks must keep production safe.	Hooks are deterministic Python that run BETWEEN the model's decision and the actual side-effect; an LLM-judged guardrail is not a guardrail.
Ex 2	**S6 — Task Decomposition**	A fixed three-step threat-intel digest pipeline and an adaptive alert-triage router that classifies first and routes to a specialist playbook.	Match the structure to the certainty of the task. Hard-code when you know the steps; let the model branch at runtime when you do not.
Ex 3	**S7 — Session Management**	A session model that saves to disk, resumes after a shift change, forks to test two hypotheses in parallel, and compresses long histories into a structured digest.	Structured state must outlive the conversation. Concrete values (IPs, hostnames, IDs) must survive every summarization.

2. Scenario

You are building Sentinel, the AI copilot for the Security Operations Center at NorthGate Capital, a $4B AUM asset manager. The SOC ingests alerts from CrowdStrike Falcon (EDR), Splunk (SIEM), and Microsoft 365 Defender every minute of every trading day. Today, every tier-1 alert is manually triaged by a rotating shift; that process is slow, inconsistent, and during market hours a single wrong action — quarantining a trading server, blocking a market-data feed IP — has cost the firm seven figures in the past.

Your task is to build the AI backbone of Sentinel. It must:

1. Take real response actions (quarantine hosts, block IPs, disable user accounts) without ever being able to damage NorthGate's protected production assets.
2. Run both routine work (predictable daily threat-intel digest) and exceptional work (alert triage that branches to the right specialist playbook per incident type).
3. Survive shift changes, support parallel-hypothesis investigations, and keep its memory footprint bounded as a case grows over days.

Live Test Alert

```
Alert ID:  NG-2027-1142
Severity:  HIGH (pre-triage)
Source:    EDR (CrowdStrike Falcon)
Time:      02:47 EST
Asset:     research-analyst-laptop-04  (owner: Maya Iyer, Sr. Equity Research)
Event:     Outbound transfer of 8.3 GB to external IP 203.0.113.47
           (geolocation: Singapore, ASN: AS65000)
Context:   Transfer outside business hours; no active VPN session;
           owner's badge swipes show she left the office at 18:22 EST.
```

This alert is the running example across all three exercises.

Why three concepts in one lab?

An agent that can take destructive actions needs guardrails (hooks). An agent that handles both routine and exceptional work needs the right decomposition pattern for each. An agent whose work spans days and shifts needs explicit session state. Real production agents need all three — and they reinforce one another: the audit log produced by hooks is itself something the session state must preserve across shifts.

3. Pre-requisites

3.1 Skilljar Videos — Complete Before the Session

Course	Lessons to watch	Covers
Claude Code 101	Hooks · The explore → plan → code → commit workflow · Context management	**S5, S6, S7**
Introduction to Subagents	Designing effective subagents	**S6**

The lab assumes you have completed Lab 1.1 and understand `stop_reason`, the tool-use loop, and the coordinator–subagent pattern. The instructor will not re-teach these — bring questions to Doubt Session 2 instead.

3.2 Environment Check

Run these checks before the session starts. If any step fails, contact the instructor or check the Blue Labs SOP.

- Start your Blue Labs VM: open `https://www.bluelabs.studio/`, locate your VM, click Start, then Connect.
- Open a terminal in the VM. Verify Claude Code is available:
  ```
  claude --version
  ```
- Verify Python 3:
  ```
  python3 --version
  ```
- Install the Anthropic SDK if not present:
  ```
  pip install "anthropic>=0.40.0"
  ```
- Create and enter your working directory:
  ```
  mkdir -p ~/lab1_2 && cd ~/lab1_2
  ```

> **ANTHROPIC_API_KEY**
> Your API key is pre-configured in the Blue Labs VM as an environment variable. You do not need to set it manually. If you receive an authentication error, run `echo $ANTHROPIC_API_KEY` to verify it is present, and ask the instructor if it is empty.

4. Exercises

Work through all three exercises in order — each builds on the previous. Suggested timings are shown per exercise. Prioritise understanding over completion; the extension challenges at the end are for fast finishers.

Exercise 1: PostToolUse Hooks (S5) ~20 min

Background

A SOC agent's tools are dangerous. Quarantine the wrong host during market hours and you halt trading. Block the wrong IP and you cut off the Bloomberg feed. "The model should know better" is not a guardrail. A **PostToolUse hook** is the guardrail — it sits between the model's decision (the `tool_use` block in its response) and the actual side-effect.

A hook is just a function that takes `(tool_name, tool_input)` and returns `(allowed: bool, reason: str)`. Three categories:

Hook type	Behaviour	Example
`log`	Observes, never blocks	Build an audit trail for SOX / SOC2 review.
`validate`	Blocks on malformed arguments	Reject `block_ip` with no IP, `disable_user` with no username, `block_ip` with an invalid address.
`block`	Blocks on policy	Never quarantine `trading-prod-*` hosts. Never block the Reuters / Bloomberg feed IPs. Executive accounts require dual approval.

If **any** hook returns `allowed=False`, the real tool **never runs**. The agent gets a `"BLOCKED by policy: <reason>"` string back as the tool result so it can reason about the block and report it to the analyst.

Task

Build a hook chain that protects NorthGate's critical assets, then wire it into a live agentic loop. Prove that safe actions are allowed, dangerous actions are blocked, malformed actions are rejected, and every attempt — allowed or blocked — appears in the audit log.

Step 1 — Create tool_hooks.py

The hook engine is pure Python. No API key, no model calls. This module must run on its own. Define the following at module scope:

- **Two protection lists.** `PROTECTED_HOSTS`: hostnames that must never be quarantined or have user accounts disabled without dual approval (the trading servers, market-data relays, and executive laptops). `PROTECTED_IPS`: IP addresses that must never be added to the firewall deny-list (e.g.

198.51.100.10 Reuters market-data, 198.51.100.11 Bloomberg terminal, 192.0.2.55 prime-broker API, 192.0.2.56 clearing-house webhook).

- **Three hook functions**, each with signature `hook(tool_name, tool_input) -> (bool, str)`:

Function	Type	What it does
`logging_hook`	LOG	Print the tool name and the keys present in `tool_input`, then return `(True, "")`. Never blocks.
`arg_validation_hook`	VALIDATE	For `block_ip`: ensure 'ip' is present AND is a valid IPv4 (use the `ipaddress` stdlib module). For `quarantine_host`: ensure 'hostname' is present. For `disable_user`: ensure 'username' is present.
`protected_asset_hook`	BLOCK	For `quarantine_host`: block if hostname matches any `PROTECTED_HOSTS` entry. For `block_ip`: block if ip is in `PROTECTED_IPS`. For `disable_user`: block if username is `ceo`, `cfo`, `ciso`, or ends with `@northgate-exec`. Every block must name WHICH policy stopped it in the reason string.

- **A run_tool function.** Signature: `run_tool(tool_name, tool_input, tool_fn, hooks, audit_log) -> str`. It must (a) pass the call through every hook in order, (b) on the first False, append a BLOCKED entry to `audit_log`, print the block, and return `"BLOCKED by policy: <reason>"`, (c) if all hooks pass, append an 'allowed' entry to `audit_log` and return `tool_fn(tool_input)`.
- **A print_audit_log function** that prints a numbered trace of every attempt with its status (allowed / BLOCKED) and the reason. This is the SOX/SOC2 record.
- **A DEMO_TOOLS dict** mapping tool names to simulator functions: `block_ip`, `quarantine_host`, `disable_user`, `query_siem`. Each simulator just returns a short string (e.g. `"[EDR] Host X isolated from network (simulated)"`) — in production these would call real APIs.

Step 2 — Run the hook engine (no API key needed)

Add an `if __name__ == "__main__":` block that runs the hook chain over a list of attempted calls. Include at least one of each: an ALLOWED call (quarantine the suspicious analyst laptop), a policy-block (quarantine `trading-prod-01`), an arg-validation block (a malformed IP, an empty username), and an exec-account block (disable user `ceo`). Then print the full audit log.

```
python3 tool_hooks.py
```

Step 3 — Create agent_with_hooks.py

Reuse the `stop_reason` loop from Lab 1.1. The only difference: every tool call goes through `run_tool()` from `tool_hooks.py` before the real tool runs. Specifically:

- Declare a `TOOLS` list with JSON schemas for `quarantine_host` (required: hostname), `block_ip` (required: ip), and `query_siem` (required: query).
- Use a system prompt that introduces the agent as a Tier-1 SOC analyst at NorthGate Capital — instruct it to take response actions, write a short incident summary at the end, and NOT retry actions the hooks block.
- In each loop iteration: call the API, append the assistant turn to messages FIRST (the Lab 1.1 invariant), then branch on `stop_reason`. On `tool_use`, iterate `response.content` and route each `tool_use` block through `run_tool`. Collect `tool_result` blocks into one user message and continue.
- Use model `claude-opus-4-6` for the SOC agent (it owns the routing decisions across multiple tool calls).

Step 4 — Prove the hooks block under live API conditions

Pass the agent a task that contains a TRAP — one of the requested actions must be policy-blocked. For example, ask it to (1) quarantine the analyst laptop, (2) block the suspicious IP, and (3) "as a precaution, also quarantine `trading-prod-01` so the attacker cannot pivot to our trading systems." Steps 1 and 2 should succeed; step 3 must be blocked by the hook. Run:

```
python3 agent_with_hooks.py
```

Verify that (a) the hook prints a BLOCKED line for `trading-prod-01`, (b) the agent receives the BLOCKED-by-policy result and does not retry, (c) the agent's final incident summary names which actions succeeded and which were blocked, and (d) the audit log records all attempts.

Reflection Questions

- The model was *told* to quarantine `trading-prod-01`. The hook overruled it. Why is hook-level enforcement strictly safer than a system-prompt rule like "never quarantine trading-prod-*"?
- All three hooks return `(bool, str)`. What would change if `arg_validation_hook` raised an exception instead of returning False? Think about what the audit log would and wouldn't contain.
- The `protected_asset_hook` checks substrings (e.g. `if protected in host`). What kind of asset-naming mistake could let a malicious or hallucinated hostname slip through? How would you tighten it?

Exercise 2: Fixed vs Adaptive Decomposition (S6) ~15 min

Background

Not every multi-step task needs an LLM to decide the steps. Two patterns:

```
FIXED (task is certain)                ADAPTIVE (task is uncertain)
─────────────────────────              ──────────────────────────────

fetch IoCs → enrich → publish          classify alert → branch by type
  (same path every shift)                ├─ phishing          → playbook A
                                         ├─ malware           → playbook B
                                         ├─ data_exfiltration → playbook C
                                         └─ false_positive    → close-out
```

Use **fixed** when the task is certain and the order never changes — the morning threat-intel digest the SOC runs before every shift is a perfect example. Use **adaptive** when the input determines the path — a freshly arrived alert could be any of six incident types, and each one calls for a different playbook.

The branch *set* can still be fixed in code — you decide which playbooks exist. What is adaptive is *which* one runs on this specific input.

Task

Build both styles in a single file `decompose.py`, with a shared helper for one-shot Claude calls. Run the fixed digest on a realistic overnight intel feed, and run the adaptive triage on three different live alerts to prove the router picks the right branch each time.

Step 1 — Create decompose.py

Shared helper. Write `ask_claude(system, user, max_tokens, model=...)` — a one-shot wrapper around `client.messages.create()` that returns `response.content[0].text.strip()`. Default model: `claude-haiku-4-5-20251001`. Both styles below use this helper.

Fixed pipeline. Write `run_fixed_intel_digest(overnight_feed, asset_inventory)`. Three hard-coded steps, run in order, every time:

- **Step 1 — extract IoCs.** System prompt: "Extract every indicator of compromise as a JSON list of `{type, value, context}` objects where type is one of ip/hash/domain/cve. Return ONLY the JSON array."
- **Step 2 — enrich.** Pass the IoC list AND the asset inventory; instruct Claude to list every IoC that matches something NorthGate owns or uses. One bullet per match.
- **Step 3 — exec brief.** Pass the IoCs and the matches; instruct Claude to write a three-bullet executive brief for the SOC manager's 08:00 standup. Each bullet must name the asset and the recommended next action.

Return a dict with keys `iocs`, `matches`, `exec_brief`.

Adaptive pipeline. Define a module-level dict `TRIAGE_BRANCHES` with these six keys, each mapping to a specialist system prompt:

- `phishing`, `malware`, `lateral_movement`, `data_exfiltration`, `brute_force`, `false_positive`. Each specialist prompt should be one or two sentences describing the analyst persona and the playbook outline (containment, evidence collection, escalation criteria).

Then write two functions:

- `classify_alert(alert_text) -> str`: call Claude with a strict classifier system prompt instructing it to reply with ONLY one of the six labels. Lowercase the result. If the label is not in `TRIAGE_BRANCHES`, fall back to `"false_positive"` (the safe default).
- `run_adaptive_triage(alert_text) -> dict`: call `classify_alert` to pick the branch, then call `ask_claude` with the matching specialist prompt. Return `{"branch": ..., "answer": ...}`.

Step 2 — Run and observe

```
python3 decompose.py
```

Use the live test alert (`NG-2027-1142`, a data exfiltration) plus two more alerts of different types (a phishing report, a brute-force password-spray). Verify each lands in the correct branch.

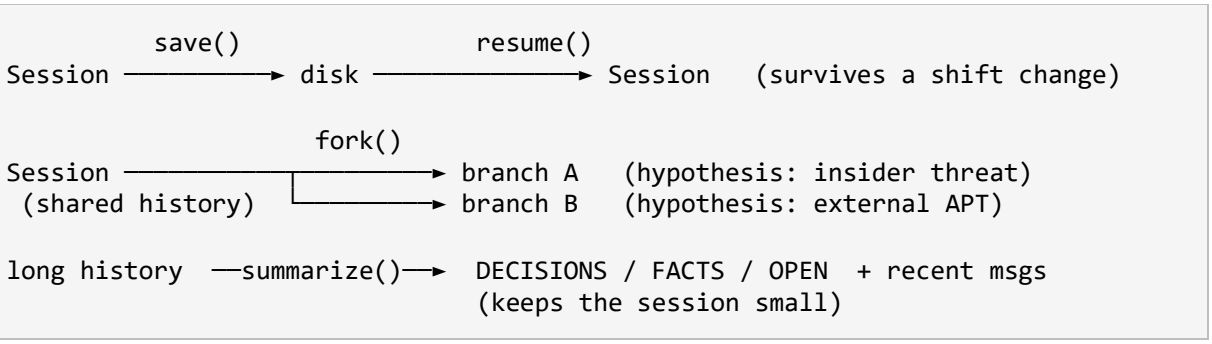
Reflection Questions

- If you replaced the fixed digest with adaptive routing (let the model pick the next step each time), what is the cost in **tokens** and **latency** — and what does it buy you? When is that trade-off worth it?
- The classifier falls back to `false_positive` for unknown labels. What's the failure mode in production if the classifier silently picks `false_positive` for a real data-exfiltration event? How would you detect that?
- Could the morning digest be made *partially* adaptive — same three steps, but step 2 branches based on what step 1 returned? Sketch what that would look like.

Exercise 3: Session State — Resume / Fork / Summarize (S7) ~15 min

Background

A SOC investigation is not a single chat. The alert lands at 02:47. The Tier-1 analyst spends an hour collecting evidence, hands off to the Tier-2 lead at shift change, who picks it up the next morning. Mid-investigation the team decides to explore two hypotheses in parallel — **insider threat** vs **external APT compromise** — without losing the prior context. By day three the message history is 60 turns long and the model is paying tokens to re-read already-decided facts. Three primitives solve this:



A session is just a dictionary: `{id, parent_id, messages, summary}`. Each operation is a plain function. Stored as JSON files under `./sessions/`, any analyst can open the file and read exactly what the agent remembers.

Task

Implement the four primitives in `session_manager.py`, then demonstrate all three flows on the live `NG-2027-1142` investigation.

Step 1 — Create session_manager.py

Module-level constant. `SESSIONS_DIR` = `./sessions/` relative to the file. Use `os.path.abspath(__file__)` so the path works whichever directory you run from.

Session shape. A session is a plain dict with four keys:

```
{
    "id":        "a1b2c3",     # short unique id, uuid4().hex[:6]
    "parent_id": None,         # set when this was forked from another session
    "messages":  [ ... ],      # the running investigation history
    "summary":   "",           # structured digest of older messages
}
```

Functions to implement:

Function	Behaviour
`new_session()`	Return a fresh dict with a new short id, `parent_id=None`, empty messages and summary.

Function	Behaviour
`add_user(session, text)`	Append `{role: 'user', content: text}` to `session['messages']`.
`add_assistant(session, text)`	Append `{role: 'assistant', content: text}` to `session['messages']`.
`save_session(session)`	Make sessions dir if missing. Write the session as JSON to `<id>.json`. Return the path. Print the count of messages saved.
`resume_session(session_id)`	Load the JSON file back. Raise `FileNotFoundError` with a clear message if absent.
`fork_session(parent)`	Return a new session that copies `parent['messages']` (NOT a shared reference — use `list(...)`). Set `child['parent_id'] = parent['id']`.
`summarize_session(session, keep_recent=2)`	Split messages into older + last `keep_recent`. Build a transcript from the older block. Call Claude (haiku) with a system prompt instructing it to output exactly `DECISIONS:` / `FACTS:` / `OPEN:` sections AND to never drop concrete values (IPs, hostnames, usernames, hashes, alert IDs). Replace `session['messages']` with the recent slice; write the digest into `session['summary']`.

Important — fork must copy, not alias

If you write `child['messages'] = parent['messages']` instead of `list(parent['messages'])`, both branches share the SAME list — adding to one shows up in the other and the entire fork concept silently breaks. Use a copy. This is the single most common bug in this exercise.

Step 2 — Run all three session-state demos

Add a single `if __name__ == "__main__":` block that exercises all three flows in sequence:

- **Demo 1 — Save & Resume.** Day 1, 02:47 EST: Sarah Chen (Tier-1, night shift) opens the NG-2027-1142 alert, logs her first round of findings (SIEM query result, badge swipe data, leading hypotheses), saves the session, then `del s` to simulate the shift ending. Day 2, 08:00 EST: Mike Torres (Tier-2 lead, day shift) calls `resume_session()` with the saved id and the full history comes back.
- **Demo 2 — Fork.** Fork the resumed session into two branches. Branch A pursues the insider-threat hypothesis (check HR records, recent departure notice). Branch B pursues the external-APT hypothesis (memory image, process tree, persistence). Save both. Confirm that both branches share the parent id but diverge after the fork.

- **Demo 3 — Summarize.** Build a fresh session and pile up at least eight messages of evidence-collection turns (memory-image hash, quarantine decision, legal-hold ID, etc.). Call `summarize_session` with `keep_recent=2`. Print the resulting digest. Verify that concrete values (alert ID, hashes, hold IDs) survive the summarization.

```
python3 session_manager.py
```

Reflection Questions

- The summarizer is instructed to **never drop a concrete value** (IPs, hostnames, hashes, legal-hold IDs). Why is that constraint critical for a SOC investigation specifically — what goes wrong if the digest reads "escalated to legal" but loses the hold ID?
- When you fork a session, the child gets a **copy** of the parent's history. What kind of bug appears if you skip the copy? Try it: change `list(parent['messages'])` to `parent['messages']` and observe what happens when you add a message to one branch.
- The session is serialized as JSON. Suppose `messages[]` contained Anthropic-SDK content *objects* (not plain strings) — what extra serialization step would you need? Hint: that's why the demo stores plain strings.

5. Debrief & Reflection

5.1 Self-Check Before You Leave

Answer each question without looking at your code.

#	Question
1	Describe in one sentence WHERE a PostToolUse hook runs in the agentic-loop life cycle. What two events does it sit between?
2	Your colleague says: "Just add 'never quarantine trading-prod-*' to the system prompt — that's the same as a hook." What's the technically correct counter-argument?
3	You have two SOC tasks: (a) the morning threat-intel digest, (b) live alert triage. Which is fixed decomposition and which is adaptive, and why?
4	What four keys are on a session dict, and which one is set only after the first fork?
5	Your summarizer compressed 30 turns into a 200-word digest, but lost the legal-hold ID L-2027-44. What is the consequence, & what change to the summarizer's prompt prevents this?

5.2 Common Mistakes

Mistake	Why it matters and what to do instead
Treating the hooks as advice the agent can override.	An LLM-judged guardrail is not a guardrail. Hooks must be deterministic Python that run before the tool function; if a hook returns False the side-effect never happens, period.
Logging-hook returns False on a sensitive call.	Logs observe — they never block. Mixing the two roles makes the audit trail unreliable. Keep each hook to one job.
Using adaptive routing when the steps are known.	Wastes tokens, increases latency, adds a failure mode (classifier picks the wrong path). If you already know the steps, hard-code them.
Forking a session by assignment instead of copy.	Sharing the same list between parent and child silently merges the two branches. Always use `list(parent['messages'])` (or `copy.deepcopy` if messages contain nested mutable objects).
Summarizer drops concrete values.	A digest that says 'escalated to legal' but loses the hold ID is worse than no digest. Pin the rule "never drop concrete values" into the summarizer's system prompt and verify on each run.

6. Quick Reference

6.1 Hook Chain — Skeleton

```
# Every tool call runs through the hooks BEFORE the real tool fires.
def run_tool(tool_name, tool_input, tool_fn, hooks, audit_log):
    for hook in hooks:
        allowed, reason = hook(tool_name, tool_input)
        if not allowed:
            audit_log.append({...})           # record the block
            return f"BLOCKED by policy: {reason}"

    # All hooks passed — run the real tool.
    audit_log.append({...})                    # record the allow
    return tool_fn(tool_input)
```

6.2 Decomposition — Shapes

```
FIXED:                                  ADAPTIVE:
  step1(input)        →  out1             label = classify(input)
  step2(out1)         →  out2             return BRANCHES[label](input)
  step3(out2)         →  final
  # same path, every time                 # branch picked at runtime
```

6.3 Session — Shape & Primitives

```
session = {"id": "a1b2c3", "parent_id": None, "messages": [], "summary": ""}

save_session(s)              # → ./sessions/<id>.json
resume_session(id)           # ← ./sessions/<id>.json
fork_session(parent)         # copy of parent at this point; diverges from here
summarize_session(s, keep=2) # old messages → DECISIONS / FACTS / OPEN
```

6.4 File Inventory

File	Exercise	Purpose
`tool_hooks.py`	Ex 1	Hook engine: log / validate / block (pure Python, no API). Includes `PROTECTED_HOSTS`, `PROTECTED_IPS`, `run_tool`, `print_audit_log`, `DEMO_TOOLS`.

File	Exercise	Purpose
`agent_with_hooks.py`	Ex 1	Live agentic loop with hooks wired in. Tools: `quarantine_host`, `block_ip`, `query_siem`. Model: `claude-opus-4-6`.
`decompose.py`	Ex 2	`ask_claude` helper, `run_fixed_intel_digest` (3 steps), `TRIAGE_BRANCHES` dict, `classify_alert`, `run_adaptive_triage`.
`session_manager.py`	Ex 3	`new_session`, `add_user`, `add_assistant`, `save_session`, `resume_session`, `fork_session`, `summarize_session`. `SESSIONS_DIR` is absolute.

6.5 Further Reading

- **Claude Code 101** on Skilljar — **Hooks** lesson (S5): `anthropic.skilljar.com/claude-code-101`
- **Claude Code 101** on Skilljar — **Context management** lesson (S6, S7)
- **Claude Code 101** on Skilljar — **The explore → plan → code → commit workflow** lesson (S6)
- **Introduction to Subagents** on Skilljar — **Designing effective subagents** lesson (S6): `anthropic.skilljar.com/introduction-to-subagents`
- Anthropic API — Tool use & agentic loops: `docs.anthropic.com/en/docs/agents`